附录B. JavaScript的编程方式

之前在附录A的综合练习中，笔者对电话交换机及其测试程序做了一系列的任务封装处理。相信细心的读者应该已经发现了，这一系列分离、组合、封装动作都来自同一个需求驱动力，那就是降低代码中的耦合度。为什么降低耦合度这件事在编程方法上占有如此重要的位置？其根本原因在于计算机程序的规模日益膨胀，以至于如今大多数程序都不是靠单打独斗就可以完成的了。既然在编程上的多人协作已经不可避免，如何进行团队分工就自然而然地成为了编程工作中首先要解决的问题。

对于分工协作，人类所有成功的经验基本上都来自于工业革命以来逐步完善的制造业工厂。为了使生产效率实现最大化，工厂生产线上所有的对象都像生产零件一样是可替换的。要想实现这一点，生产线上各个对象之间的耦合度就必须降到最低，理论上它们都只需要做好自己的事情，彼此甚至都不需要知道对方的存在。而致力于将硬件生产线上的成功经验复制到编程工作中来，形成软件生产线的努力就被我们称之为“软件工程”，正是这种将软件生产工程化的努力催化出了致力于从各个角度降低耦合度的编程方法。在这篇附录中，我们将为读者介绍面向对象编程与异步编程这两种编程方法在JavaScript中的使用和具体实现方式。

B.1 面向对象编程

在程序被分割成一个个对象，并且将对象之间的耦合度降低到一定程度之后，其开发者们就会面临随之而来的一个新问题，那就是这些对象之间要如何互动呢？例如在之前的示例中，当电话交换机的实现被封装成一个对象之后，我们随即针对性地为该对象编写了如下测试代码。

```
let phoneExch = new TelephoneExchange(['张三', '李四', '王五', '赵六']);
phoneExch.callAll();
console.log('-----------');
phoneExch.add('owlman');
phoneExch.callAll();
console.log('-----------');
phoneExch.delete(1002);
phoneExch.callAll();
console.log('-----------');
phoneExch.update(1003,'batman');
phoneExch.callAll();
console.log('-----------');
```

如读者所见，这段代码测试了电话交换机的初始化，电话线路的添加、删除与修改，并且在每次线路变化时都会呼叫一遍所有线路，以便确认其是否工作正常。但是，这些测试都只是针对`TelephoneExchange`这个类的对象来编写的。我们现在要思考的是：如果电话交换机有了新的实现方式，其测试代码是否也要随之重写呢？更重要的问题是，如果我们是要对新的电话交换机实现进行完全不一样的测试，重写当然是很自然的事，但对于电话交换机来说，除了线路网络的初始化，增加、删除、修改与呼叫之外，还有多少新的测试可写呢？既然所有的电话交换机测试都大同小异，上面的测试代码为什么就不能被重用呢？

要想找到这个问题的解决思路，我们就需要先回到软件工程的思想源头，看看人们在工厂的生产线上是怎么解决类似问题的。在关于流水线生产的描述上，很多人应该都听说过“流水线上所有对象都是一颗可被替换的螺丝钉”的说法。是的，这就是流水线生产的最理想状态。对于一颗螺丝钉，若想成本低廉，它可以是铁的，若不想它生锈，它可以是不锈钢的，若想减轻重量，它可以是铝合金的。但无论这些螺丝钉是用什么做的，它们最终都必须要能被拧进一个螺丝孔里。换句话说，在工厂流水线上，判断螺丝钉是否可相互替换的测试方法只有一

个，那就是看它们能不能被拧进被指定的螺丝孔里，这与它的材质并没有多大的关系。对应到编程术语中，用不锈钢、铝合金这些材质制作螺丝钉的过程被称作对象的“实现”，而这些螺丝钉最终要被拧进去的这个螺丝孔则被称作对象的“接口”。所以从此可以看出，让上述测试代码可被重用的关键就在于我们能否实现接口一致的电话交换机。对于这种将对象接口一致化的编程方式，我们通常称之为**面向对象编程**。

在面向对象编程的方法论中，我们通常会先为将要执行某一类操作的对象设计一个抽象的“基类”，然后以该类的接口为基准来实现其他用于具现化这些对象的“具体类”。在编程术语中，我们称这些后来的具体类是“继承”了基类的接口，它们是这个基类的“子类”，因而基类有时也被称作“父类”。接下来，就让我们来看看具体到了JavaScript中，父类、子类以及它们之间的接口继承是如何实现的。

B.1.1 接口设计与实现

在具体设计并实现对象的接口之前，我们首先要了解如何用编程语言来实现“接口”这个概念。相信学习过Java的读者应该都还记得，我们在使用这些传统面向对象编程语言定义类的时候，通常都会被要求为其属性和方法设置`private`、`public`等不同层次的访问权限。通常情况下，被设置为`private`的属性和方法主要供其所在对象内部来使用，对象外面的代码是无法直接使用它们的。而被设置为`public`的属性和方法则是直接提供给该对象的调用方来使用的。在编程术语上，被设置为public的这部分属性和方法通常就被称之为**接口**。

所以，设计对象接口的工作本质上就是在决定对象的哪一部分应该提供给调用方，成为接口，而哪一部分应该被视为对象的实现细节而隐藏在对象内部。但这样一来，我们就会立即面临到一个现实问题：JavaScript在语法层面上并没有提供`private`、`public`这种限制权限的机制。为此，我们得使用一些变通手段来完成接口的设计工作，下面就来为介绍两种目前较为常用的解决方案。

- **编码规范约束**：众所周知，编码规范是程序员之间靠自觉遵守的一种君子协议。它虽然不具有强制性，但为了使团队协作顺畅，降低代码维护成本，大部分程序员都会选择自觉遵守。所以，对象的设计者可以通过某种约定的命名方式告诉对象的调用方，哪一些属性和方法是不希望被对象外部的代码使用的。例如对于一个`Point`类的对象来说，如果我们不希望外界直接访问它的`x`、`y`属性，就可以像下面这样，在这些属性名前面加上一个下划线。

  ```
  class Point {
      constructor(x,y) {
          this._x = x;
          this._y = y;
      }

      printCoords() {
          console.log('坐标：（'+ this._x + ', ' + this._y + ）');
      }
  };
  ```

  这样一来，我们就等于告诉了`Point`类的调用方一个信息：“`_x`和`_y`是私有属性，为了相关代码的日后维护，请不要直接使用它们”。当然，必须再强调一次，这种约束没有强制性，如果有人没有团队协作概念，硬要像下面这样，直接使用甚至修改这两个属性，JavaScript解释器是不会报错的。

  ```
  const p = new Point(5,5);
  p.printCoords();  // 输出：坐标：（5,5）
  p._y = -10;
  p.printCoords();  // 输出：坐标：（5,-10）
  ```

- **局部变量方案**：在JavaScript中，由于变量的作用域被分为了函数作用域和全局作用域（ES6标准新增了块级作用域），函数内部的局部变量在函数之外是不可见的，所以我们可以利用这一特性将需要设置为`private`的属性隐藏在构造函数的作用域内。例如，对于上面的`Point`类，我们也可以像下面这样定义。

```
class Point {
    constructor(x,y) {
        let _x = x;
        let _y = y;

        this.printCoords = function() {
            console.log('坐标：（'+ _x + ', ' + _y + ''）');
        };
    }
};

const p = new Point(5,5);
p.printCoords();   // 输出：坐标：（5,5）
p._y = 10;         // 操作无效，只是给 p 对象额外增加了一个 _y 属性
p.printCoords();   // 输出：坐标：（5,5）
```

  如读者所见，`Point`类的调用方现在无法修改直接访问类定义的坐标属性了，它只能为`p`这个单一对象添加一个无用的_y属性，影响不了对象方法的行为。但这样做的坏处也是显而易见的：首先，为了让`Point`类自己的方法能使用构造函数的局部变量，我们得用属性的形式将这些方法定义在构造函数内部，这不仅会造成之前中所说的代码冗余，而且也破坏了ES6标准新增的类定义语法，使我们在很大程度上用回了ES5时代以及更早之前的、那种不够优雅且隐晦难懂的语法。但无论如何，这种解决方案是真正实现了将相关对象属性设置为`private`的效果，不再需要调用方自觉遵守某种约定了。

总而言之，虽然JavaScript本身没有提供私有化对象属性和方法的机制，但我们可以利用这门语言强大的自由度来实现类似的功能。但我们同时也必须谨记“自由即责任”的原则，在享受自由度带来的强大功能之时，也要承担自由所带来的风险，这实际上也是对程序员自身能力的严峻考验。

在隐藏了对象的“实现”部分之后，我们接下来就该来考虑对象的“接口”问题了。这个问题的另一个问法是：我们希望调用方如何使用对象？例如，对于上面的`Point`类，调用方只能在初始化对象时为其指定一个二维标，然后就只能通过`printCoords`方法来在终端输出该“点”对象的坐标了，所以我们可以理所当然地认为，`Point`类的设计者希望提供的是一个固定的“点”对象，它在坐标系中的位置是不变的。接下来，如果我们将`Point`类的设计意图修改成一个可移动的“点”对象，就必须像下面这样，为其调用方提供一个修改坐标的接口。

```
class Point {
    constructor(x,y) {
        let _x = x;
        let _y = y;

        this.printCoords = function() {
            console.log('坐标：（'+ _x + ', ' + _y + ''）');
        };

        this.updateCoords = function(x,y) {
```

```
            _x = x;
            _y = y;
        };
    }
};


const p = new Point(5,5);
p.printCoords();          // 输出:  坐标:  (5,5)
p.updateCoords(10,10);
p.printCoords();          // 输出:  坐标:  (10,10)
```

如读者所见，现在`Point`类的调用方可以通过`updateCoords`方法来修改“点”对象的坐标了。而且，从这里读者也可以看出允许直接修改`_x`、`_y`属性，与让人通过接口来修改坐标的区别，那就是我们可以通过后者来限制调用方的动作，以防止一些破坏性的修改。例如，如果我们的“点”对象只能在坐标系第一象限内活动，那就必须确保`_x`和`_y`不能为负值，为此我们需要修改一下`updateCoords`方法的实现，具体如下。

```
this.updateCoords = function(x,y) {
    if ((x<0) || (y<0)) {
        console.log('坐标不能为负值! ');
        return false;
    }
    _x = x;
    _y = y;
};
```

这样一来，当该类的调用方执行`p.updateCoords(10,-10)`这样的操作时，修改动作就会被制止。同样地，如果我们还希望允许调用方以只读方式获取`_x`或`_y`坐标值，还可以再提供`getX`和`getY`这两个方法。具体如下。

```
class Point {
    constructor(x,y) {
        let _x = x;
        let _y = y;

        this.printCoords = function() {
            console.log('坐标:  ('+ _x + ', ' + _y + ') ');
        };

        this.updateCoords = function(x,y) {
            if ((x<0) || (y<0)) {
                console.log('坐标不能为负值! ');
                return false;
                }
            _x = x;
            _y = y;
        };

        this.getX = function() {
            return _x;
```

```
        };

        this.getY = function() {
            return _y;
        };
    }
};

const p = new Point(5,5);
p.updateCoords(10,10);
console.log(p.getX()+','+p.getY()); // 输出: 10,10
```

现在，相信读者已经对接口的设计思路有了一个初步的认识，要想更深一步理解接口的作用，就必须要了解如何让未来的类共享现有类的接口，以实现真正的面向对象编程了。

B.1.2 使用类继承语法

希望上一节中那些自由放纵的，新旧标准混搭的类定义语法没有给读者带来太大的困惑。毕竟，在笔者个人看来，JavaScript这门语言最大的魅力就在于“自由”，当然享受自由这件事需要量力而行，不要忘记“自由即责任”的原则。接下来，为了让读者对类型继承能有一个较为直观的理解，我们会先暂且用回ES6标准制定的类定义语法，以便引出对类继承的介绍。待掌握类型继承机制在JavaScript语言中的具体实现之后，我们再来讨论如何将上述隐藏实现的技巧也应用到类型继承机制中。

书归正传，想必大家都已经知道了，ES6 标准为JavaScript语言新增了与传统面向对象语言相类似的类型继承语法，具体如下。

```
class [子类名] extends [父类名] {
    constructor([构造形参]) {
        super([父类的构造实参]);
        [创建子类属性并将其初始化]
    }

    [子类方法定义]
};
```

下面，我们来对类继承语法中的各个单元做个说明：首先，该语法依然是一个类定义语句，所以它沿用了之前类定义语句中所有的元素。只不过，这次我们是基于一个现有类来构造新的类型，它必须用`extends`关键字为自己指定一个基础类型，所以这里的`[父类名]`必须是一个已经被定义的类名。然后，在子类的`constructor`函数中，我们必须先用`super`关键字调用父类构造函数，然后才能用`this`引用添加子类的属性，因为在类继承语法中，子类的`this`是在父类构造函数中创建的。例如，如果我们想基于之前的`Point`类创造一个可以标记颜色的“点”类型`colorPoint`，就可以像下面这样做。

```
class Point {
    constructor(x,y) {
        this._x = x;
        this._y = y;
    }
```

```
    printCoords() {
        console.log('坐标: ('+ this._x + ', ' + this._y + ') ');
    }

    updateCoords(x,y) {
        if ((x<0) || (y<0)) {
            console.log('坐标不能为负值! ');
            return false;
            }
        this._x = x;
        this._y = y;
    };

    getX() {
        return this._x;
    };

    getY() {
        return this._y;
    };
};

class colorPoint extends Point {
    constructor(x,y,color) {
        super(x,y);
        this._color = color;
    }

    updateColor(color) {
        this._color = color;
    }

    printCoords() {
        super.printCoords();
        console.log('颜色: ', this._color);
    }
}

const p = new colorPoint(5,5,'红');
p.printCoords();         // 输出: 坐标: (5,5)
                                 //            颜色: 红
p.updateCoords(10,10);
p.updateColor('绿')
p.printCoords();         // 输出: 坐标: (10,10)
                                 //            颜色: 绿
```

从上述代码中，我们可以看到以下三点。

- 第一：子类可以原封不动地共享父类的接口及其实现，譬如`updateCoords`方法就完全使用了父类原有的实现。
- 第二：子类可以共享父类的接口并修改其实现，譬如`printCoords`方法的实现就在子类中被重新修改了。

- 第三：子类可以在父类实现的基础上为自己添加新的属性和方法，譬如`_color`属性和`updateColor`方法就是子类中新增的部分。

在编程术语上，上述这种子类共享父类接口与实现的方式就被称作“继承”，而对于继承自父类的接口进行重新实现的行为，我们通常称之为“重写”或“覆写”。请注意，从上面我们对`printCoords`方法的覆写可以看出，即使该方法被覆写了，父类的`printCoords`方法在子类的内部依然是可用的，我们可以通过`super`关键字来调用它。

需要注意的是，`super`关键字在子类中实际上有两种用法：第一种是使用函数调用操作符，例如`super()`，这种用法只能用于调用父类的构造函数，并且只能在子类的构造函数中调用。第二种是使用成员操作符，例如`super.printCoords()`，这种用法只能用于在子类方法中调用父类的方法。是的，很令人意外，我们“只能”用`super`调用父类的方法，而不能用它来获取父类的属性。也就是说，如果我们在子类方法中调用`console.log(super._x)`，得到的会是`undefined`。这是为什么呢？这就涉及到JavaScript中对象的具体实现问题了。

B.1.3 深度探索对象

众所周知，在ES6标准发布之前，在JavaScript中是用构造函数来充当“类”的角色的。那么，构造函数与它所创建的对象之间究竟是什么关系呢？事实上，这个问题也可以回到工厂生产过程中去寻找答案。大家都知道“类”的作用就相当于是对象的模版，现在请试想一下，如果要在没有模版的情况下生产一个新产品，工人们会怎么做呢？答案是，他们通常会先试着找到一个现有产品，然后再以该产品为基础设计出一个原型产品，最后再以原型产品为基准来生产目标产品。对应在编程术语上，这种用作基准的对象被称作目标对象的“原型对象”，而构造函数的作用就是为将要创建的对象设计原型对象。

B.1.3.1 使用原型对象

换而言之，在JavaScript中，每个对象都会有一个原型对象，而其原型对象则来自于它们各自的构造函数。那么，这个说法可以用代码来证明吗？答案是肯定的，我们可以通过JavaScript的内置对象方法`Object.getPrototypeOf()`来查看指定对象的原型对象，下面来做个实验：先定义一个构造函数，然后再用它创建两个对象，看看这两个对象的原型是什么。

```
function Hero(name) {
    this.name = name;
}

let hero_1 = new Hero('owlman');
let hero_2 = new Hero('batman');

console.log(hero_1.name)                      // 输出:  owlman
console.log(Object.getPrototypeOf(hero_1));  // 输出:  Hero ()
console.log(hero_2.name)                      // 输出:  batman
console.log(Object.getPrototypeOf(hero_2));  // 输出:  Hero ()
```

从上述实验的结果，我们可以看到：`hero_1`和`hero_2`的`name`属性值分别为`"owlman"`和`"batman"`，但它们的原型对象都指向了构造函数`Hero()`。这也就是说，这两个对象虽然有各自独立的属性，但却是基于同一个原型对象的产物，而这个原型对象正是它们的构造函数。另外，这个实验也间接证明了我们之前所说的：如果在构造函数中将对象的方法也定义在`this`引用上，就会让每个实体对象都拥有一份独立但相同的方法实现，这意味着，如果我们用构造函数创建一万个对象，同一个对象方法就会被重复实现一万次，这显然会造成代码冗余。

所以，一个对象的方法实现应该被定义在其原型对象上，这样只要对象来自同一个原型对象，它们调用的就是同一份方法实现。

但我们都知道，其实构造函数本身在没有被new操作符调用之前只是一个普通函数，而在JavaScript中，函数本身也是一个对象，它又是如何在被new操作符调用时成为了其所创建对象的原型对象的呢？答案就是函数的prototype属性。在JavaScript中，为了让用户定义的函数都可以被当作构造函数来使用，每个由用户定义的函数都会有一个prototype属性。当这些函数被当作普通函数调用时，该属性是被忽略的，然而一旦它们被new操作符调用，该属性就会被初始化，并成为了其所创建对象的原型对象。例如，如果我们想为上面创建的hero_1和hero_2对象添加一个sayHello方法，又不想有代码冗余的问题，就可以接着上面的实验代码继续编写如下代码。

```
Hero.prototype.sayHello = function() {
    console.log('Hello,', this.name);
};                          // 在原型对象上添加方法

hero_1.sayHello();          // 输出:  Hello, owlman
hero_2.sayHello();          // 输出:  Hello, batman

Hero.prototype.sayHello = function() {
    console.log('你好,', this.name);
};                          // 修改原型对象上的方法

hero_1.sayHello();          // 输出:  你好, owlman
hero_2.sayHello();          // 输出:  你好, batman

hero_1.sayHello = function() {
    console.log(this.name, '不是英雄! ');
};                          // 在 hero_1 对象上添加同名方法

hero_1.sayHello();          // 输出:  owlman不是英雄!
hero_2.sayHello();          // 输出:  你好, batman

delete hero_1.sayHello;     // 删除 hero_1 上的同名方法

hero_1.sayHello();          // 输出:  你好, owlman
hero_2.sayHello();          // 输出:  你好, batman
```

如读者所见，如果在构造函数的prototype属性上添加或修改方法的实现，会影响其创建的所有对象，而如果我们在某个实体对象上添加或修改方法的实现，不会影响到其他对象。另外，上面的代码也说明了一件事，即当一个对象实体的this引用上的方法与其原型对象上的方法同名时，JavaScript 解释器会优先调用this引用上的方法。这也是我们不赞成在构造函数中用this引用来定义对象方法的另一个原因，它会隐藏掉我们在原型对象上定义的方法。

B.1.3.2 再探对象属性

除了方法之外，属性也是可以被定义在原型对象上的。在JavaScript中，对象的属性按其拥有者可分为被定义在this引用上的自有属性和被定义在原型对象上的原型属性两种。对此，我们可以使用每个对象都拥有的hasOwnPrototype方法来判断一个属性是否为当前对象的自有属性。下面，让我们接着上面的实验继续编写如下代码。

```
Hero.prototype.counter = 2;                      // 添加一个原型属性
console.log(hero_1.hasOwnProperty('name'));      // 输出: true
console.log(hero_1.hasOwnProperty('counter'));   // 输出: false
console.log(hero_1.name === hero_2.name);        // 输出: false
console.log(hero_1.counter === hero_2.counter)   // 输出: true
```

从上述实验的结果，我们可以看到：由于name属性是被构造函数定义在this引用上的，所以hero_1.hasOwnProperty('name')返回的是true，而新增的counter属性则是被定义在Hero.prototype上的，所以hero_1.hasOwnProperty('counter')返回了false。另外，对象的自有属性是彼此独立的，而原型属性则是该原型所创建的所有对象共同拥有的。所以在原型对象上添加属性需要格外小心，该属性会影响到该原型对象所创建的所有对象，包括在它被添加到原型对象之前所创建的对象。接下来，我们可以搭配用于枚举对象属性的for-in循环来看看hero_1对象中到底有哪些自有属性和原型属性。

```
for(property in hero_1) {
    if (hero_1.hasOwnProperty(property)) {
        console.log('自有属性: ', property);
    } else {
        console.log('原型属性: ', property);
    }
}
// 以上代码输出:
// 自有属性:  name
// 原型属性:  sayHello
// 原型属性:  counter
```

如读者所见，for-in循环的作用是遍历指定对象中所有可枚举的属性，程序会依次将hero_1对象中的元素读取到循环变量（即这里的property变量）中，然后交由hasOwnProperty方法来判断是否为自有属性。当然，在这里我们会发现一件有趣的事，那就是sayHello方法也会被当作属性被枚举了出来，这本身倒不难理解，毕竟它可以被看作是值为函数的属性。比较让人难以理解的是：既然对象方法也会被当作属性被列举出来，我们明明眼见hero_1对象调用了hasOwnProperty方法，为什么该方法却没有被列出来呢？事实上，这个问题涉及到了JavaScript中的最终原型对象Object，我们将会在下一节中详细介绍该对象。现在，读者先暂且只需要知道for-in循环遍历的是可枚举的属性，而来自Object中的原型属性都是不可枚举的。在JavaScript中，每个对象的属性都有属性值、可写性、可枚举性以及可配置性四个特性，它们分别对应着以下四个“属性描述符”：

- value：即属性的值，这个特性我们已经一直在使用了，它决定的是属性中存储的数据，默认值为undefine。
- writable：即属性是否可写，默认值为true、如果将其设置为false，该属性在初始化之后就不可再被修改了，请注意，这里的“不可修改”针对的是所有地方，包括该属性所在对象的方法，这与我们之前讨论的“让对象外部不能直接修改属性"不是一回事。
- enumerable：即属性是否可枚举，默认值为true。如果将其设置为false，该属性就不会被for-in循环遍历到。
- configurable：即属性是否可配置，默认值为true。如果将其设置为false，该属性在初始化之后其所有的特性，包括congfigurable特性本身，就都不可修改了。

如果我们想查看现有属性的特性配置，我们可以使用Object.getOwnPropertyDescriptor()方法来查看。例如，如果想查看上述实验中hero_1对象的name属性，我们就可以接着上面的代码这样写：

```
let msg = Object.getOwnPropertyDescriptor(hero_1, 'name');
console.log(msg);
// 以上代码输出:
// { value: 'owlman',
//   writable: true,
//   enumerable: true,
//   configurable: true }
```

在这里，我们需要传递给`Object.getOwnPropertyDescriptor()`方法两个实参：第一个实参是我们要查看属性所属的对象，第二个实参是该属性的名称，请注意，该名称必须用一个字符串来表示。需要说明的是，在通常情况下，我们在定义对象属性时只需要指定它的`value`即可，其它三个特性会被自动被赋予默认值。当然了，如果实在有特定的需求，我们也可以使用`Object.defineProperty()`方法来详细定义属性。换句话说，下面两种定义属性的方式在效果上是完全一致的。

```
hero_1.test = 'test';
console.log(hero_1.test);          // 输出:  test
Object.defineProperty(hero_1,'test', {
    value: 'test',
    writable: true,
    enumerable: true,
    configurable: true
});
console.log(hero_1.test);          // 输出:  test
```

在这里，我们需要传递给`Object.defineProperty()`方法三个实参：第一个实参是我们要定义属性所属的对象；第二个实参是该属性的名称，该名称必须用一个字符串来表示；第三个实参是一个用于逐条指定该属性特性的对象直接量。从这里我们也可以看出，事实上每个对象的属性本身也是一个对象。除此之外，在`configuration`没有被设置为`false`的前提下，我们也可以用`Object.defineProperty()`方法来修改现有属性的特性。例如，如果我们不想让`for-in`循环遍历到`test`属性，就可以接着上面的代码继续编写如下代码。

```
Object.defineProperty(hero_1,'test', {
    value: 'test',
    enumerable: true
});
```

到目前为止，我们一直在对`Hero()`构造函数所创建的对象执行各种扩展、修改和缩减操作，充分展现了JavaScript对象的灵活性和自由度，但正如我们一直所强调的，享受自由的同时必须注意随之而来的风险。在某些情况下，如果我们觉得不能放任自己设计的对象像上面这样被调用方随意增加、修复甚至删除属性，就可以对这些行为进行禁止。在JavaScript中，禁止修改对象的方法有三个，下面我们就逐一来介绍一下它们。

首先，我们可以用`Object.preventExtensions()`方法将它们设置为不可扩展的对象。这时候，如果我们用`Object.isExtensible()`方法来查看它们的可扩展性，就会看到其返回`false`了。下面，我们就继续拿已经饱受折磨的`hero_1`对象来试一下。

```
console.log(Object.isExtensible(hero_1));  // 输出:  true
Object.preventExtensions(hero_1);
console.log(Object.isExtensible(hero_1));  // 输出:  false
hero_1.isbaby = true;
console.log(`isbaby` in hero_1); // 输出 false, 证明操作失败
```

如读者所见，`isbaby`属性已经不能被添加到`hero_1`中，如果声明为严格模式，上述代码还会直接报错，并被JavaScript解释器终止执行。需要提醒的是，`Object.preventExtensions()`方法执行的操作是不可逆的，`hero_1`一旦被设定为不可扩展，就不可能再被改回来了，所以对于这个操作，程序员在执行之前可要想好了。那么，`hero_1`现在是不是终于可以松口气，庆祝一下它被我们反复折腾的恶梦结束了呢？如果它真的这么想，那就太天真了。`Object.preventExtensions()`方法只能保证对象的自有属性不会再被扩展了，但我们仍然可以删除它的自有属性。例如，如果我们想删除之前添加的`test`属性，就可以像下面这样做。

```
console.log(`test` in hero_1); // 输出 true, 证明该属性目前存在
delete hero_1.test;
console.log(`test` in hero_1); // 输出 false, 证明删除操作成功
```

然后，如果不想允许调用方删除对象的属性，我们还可以使用`Object.seal()`方法封印对象。同样地，在执行封印操作之后，我们可以用`Object.isSealed()`方法来验证封印是否成功。在一个对象被封印之后，它不但无法再扩展新的自有属性，现有的自有属性也会被设置为不可配置。这样一来，如果我们再想删除`hero_1`对象的`name`属性，操作就会失败，例如像下面这样。

```
console.log(Object.isSealed(hero_1)); // 输出:  false
Object.seal(hero_1);
console.log(Object.isSealed(hero_1)); // 输出:  true
console.log(`name` in hero_1); // 输出 true, 证明该属性目前存在
delete hero_1.name;
console.log(`name` in hero_1); // 输出 true, 证明删除操作失败
```

请注意，封印对象的操作也是不可逆的，对象一旦被封印就无法再解封了。现在，`hero_1`对象的自有属性不能被添加或删除，也无法再配置可写性和可枚举性了，但我们仍然可以修改它的`name`值。例如，我们可以接着上面的代码这样写。

```
hero_1.name = 'owlbaby';
hero_1.sayHello();          // 输出:  你好, owlbaby
```

这时候，`hero_1`对象的状态就基本类似于我们在C++、Java这些语言中使用的对象了，由此也可以看出JavaScript相对于这些传统的面向对象编程语言给予了程序员多大的自由度。当然，如果我们想彻底将`hero_1`设置成一个不可修改的对象，我们还可以使用`Object.freeze()`方法冻结对象，同样地，在冻结了对象之后，我们可以使用`Object.isFrozen()`来验证操作是否成功。在`hero_1`对象被冻结之后，如果我们再想修改其`name`属性，操作就会无效，在严格模式下，JavaScript解释器还会报错并终止执行。

```
console.log(Object.isFrozen(hero_1));  // 输出： false
Object.freeze(hero_1);
console.log(Object.isFrozen(hero_1));  // 输出： true
hero_1.name = 'owlman';
hero_1.sayHello();         // 输出“你好，owlbaby”， 证明操作失败。
```

同样地，冻结对象也是不可逆的，一旦对象被冻结了，他就无法再被解冻。现在，`hero_1`可以稍微安心一点了，它被我们反复折腾的日子总算基本结束了。当然了，我们还是可以通过`Hero.prototype`来为其添加新的方法，但这个操作同时也会影响到`hero_2`和其他用`Hero()`构造函数创建的对象，甚至还包括继承了`Hero.prototype`原型的后续原型对象，以及它们所创建的对象，程序员们至少会更谨慎一些。为了让读者能对这一部分的操作做出合理的决策，我们接下来就要来详细介绍对象之间，原型之间的继承关系。

B.1.3.3 理解`Object`对象

细心的读者可能已经发现了，我们之前在介绍原型对象时，似乎遗漏了一个关键问题：既然JavaScript中的每个对象都有自己的原型对象，而用构造函数创建的对象的原型对象来自于其构造函数，那么充当构造函数的这些函数的原型对象是什么呢？这些原型对象的原型对象又是什么呢？以此类推的话，这个问题似乎可以一直问下去，但这显然是不现实的，所以我们可以推断出JavaScript所定义的世界中必定有一个最初的原型对象，所有对象的原型都来自于它，这个对象就是我们之前一直在使用，但还尚未做说明的`Object`对象，它的`Object.prototype`属性是JavaScript中所有对象的原型。如果我们用直接量创建一个对象，它的原型就是`Object.prototype`。对此，我们可以用以下方法来查看一下。

```
let machine = { name:'robot'}; // 用直接量创建对象
let prototype = Object.getPrototypeOf(machine);
console.log(prototype === Object.prototype); // 输出： true
```

这也就是说，当我们用直接量创建对象的时候，实际上就是调用了`Object`构造函数，即：

```
// 当我们这样写：
let machine = { name:'robot'};
// 就等同于：
let machine = new Object();
machine.name = 'robot';
```

在JavaScript中，`Object`是一个极为特殊的构造函数，它不但提供了可由JavaScript中所有对象调用的实例方法，还提供了一组只能由`Object`本身调用的静态方法。虽然我们之前或多或少都已经使用过这些方法了，但始终没有系统地介绍过它们。下面，就让我们来补上这一课。

实例方法：

`Object`构造函数提供的实例方法主要有六个，鉴于`Object.prototype`是JavaScript中所有对象的原型，这些方法应该可以被所有对象调用，下面是这些方法的具体介绍。

- `hasOwnProperty(propertyName)`方法：用于判断其调用对象中是否有名为`propertyName`的自由属性，是就返回`true`，不是则返回`false`。例如，对于上面的machine对象，我们可以像下面这样判断其

name属性是否为自由属性。

```
console.log(machine.hasOwnProperty('name'));
// 输出： true
```

- propertyIsEnumerable(propertyName)方法： 用于判断其调用对象的propertyName属性是否属于可枚举属性，是就返回true，不是则返回false。例如，对于上面的machine对象，我们可以像下面这样判断其name属性是否为可枚举属性。

```
console.log(machine.propertyIsEnumerable('name'));
// 输出： true
```

- isPrototypeOf(object)方法： 用于判断其调用对象是否为object对象的原型对象，是就返回true，不是则返回false。例如像下面这样。

```
console.log(Object.prototype.isPrototypeOf(machine));
// 输出： true
```

- toString()方法： 用于返回其调用对象的、被本地化了的字符串描述。例如像下面这样。

```
let arr = [1, 2, 3];
console.log(arr.toString()); // 输出： 1,2,3
```

- toLocaleString()方法： 用于以用户所在系统设置的语言返回其调用对象的字符串描述。例如像下面这样。

```
let date = new Date();
console.log(date.toLocaleString());
// 输出： 2019/10/6 下午3:26:21
```

- valueOf()方法： 用于返回其调用对象的原始值。例如像下面这样。

```
let arr = [1, 2, 3];
console.log(arr.valueOf());  // 输出：  [ 1,2,3 ]
```

静态方法：

Object的静态方法指的是只能由Object自身作为一个实体来调用的方法，这些方法主要用于更细致、更精确地创建、使用和修改JavaScript中的对象。下面是这些方法的具体介绍。

- `Object.assign(target, ...sources)`方法： 用于将`sources`指定的一个或多个对象复制到`target`对象中，并将其作为一个新创建的对象返回。在复制过程中，如果`target`对象与`sources`对象有同名属性，后者的属性会覆盖前者。如果`sources`中的多个对象有同名属性，则靠后出现的属性会覆盖前面的属性。

  需要提醒的是，Object.assign()方法执行的是浅拷贝而非深拷贝，所以如果sources对象中的某个属性值是对象类型的，那么target对象得到的是该属性值的引用，例如像下面这样。

  ```
  let obj_1 = { x: { num: 1 } };
  let obj_2 = Object.assign({}, obj_1);
  console.log(obj_2.x.num); // 输出： 1
  obj_1.x.num = 2;
  console.log(obj_2.x.num); // 输出： 2
  ```

- `Object.create(prototype, propertyDescriptor)`方法： 用于以`prototype`指定的对象为原型创建一个对象，并赋予其用`propertyDescriptor`描述的属性。该方法有利于我们更精确地创建一个对象，例如像下面这样。

  ```
  // 对于下面这个对象：
  let machine = { name:'robot'};
  // 我们也可以这样定义：
  let machine = Object.create(Object.prototype,{
      name: {
          value: 'robot',
          writable: true,
          enumerable: true,
          configurable: true
      }
  });
  ```

- `Object.defineProperty(target, propertyDescriptor)`方法： 用于将`propertyDescriptor`描述的属性添加到`target`对象中，如果该属性已经存在，就覆盖它。由于我们之前已经演示过该方法的使用，这里就不再举例了。

- `Object.defineProperties(target, propertyDescriptor)`方法： 用于将`propertyDescriptor`描述的多个属性添加到`target`对象中，如果这些属性已经存在，就覆盖它们。例如像下面这样。

  ```
  Object.defineProperties(machine,{
      name: {
          value: 'robot',
          writable: true,
          enumerable: true,
          configurable: true
      },
      cpu: {
          value: 'i5',
          writable: true,
  ```

```
        enumerable: true,
        configurable: true
    }
});

console.log(machine.name, machine.cpu); // 输出 robot i5
```

- `Object.entries(target)`方法： 用于以键/值对数组的形式返回`target`对象的可枚举属性。例如，对于上面的`machine`对象，我们可以像下面这样查看它的可枚举属性。

```
console.log(Object.entries(machine));
// 输出： [ [ 'name', 'robot' ], [ 'cpu', 'i5' ] ]
```

- `Object.keys(target)`方法： 用于以数组的形式返回`target`对象中可枚举属性的名称。例如，对于上面的`machine`对象，我们可以像下面这样来查看其各项可枚举属性的名称。

```
console.log(Object.keys(machine));
// 输出： [ 'name', 'cpu' ]
```

- `Object.values(target)`方法： 用于以数组的形式返回`target`对象中可枚举属性的值。例如，对于上面的`machine`对象，我们可以像下面这样查看其各项可枚举属性的值。

```
console.log(Object.values(machine));
// 输出 [ 'robot', 'i5' ]
```

- `Object.getOwnPropertyDescriptor(target, propertyName)`方法： 用于获取`target`对象的`propertyName`属性的具体描述。例如，对于上面的`machine`对象，我们可以像下面这样来查看它的`name`属性。

```
console.log(Object.getOwnPropertyDescriptor(machine,'name'));
// 以上代码输出:
// { value: 'robot',
//   writable: true,
//   enumerable: true,
//   configurable: true }
```

- `Object.getOwnPropertyNames(target)`方法： 用于以一个字符串数组的形式返回`target`对象中所有自有属性的名称。例如，对于上面的`machine`对象，我们可以像下面这样来查看它的自由属性。

```
console.log(Object.getOwnPropertyNames(machine));
// 以上代码输出： [ 'name', 'cpu' ]
```

- `Object.getOwnPropertySymbols(target)`方法： 用于以一个数组的形式返回`target`对象自身所有的符号属性。由于我们目前还没有介绍ES6新增的`Symbols`对象，暂时就先不举例了。
- `Object.getPrototypeOf(target)`方法： 用于返`target`对象的原型对象。由于我们之前已经演示过该方法的使用，这里就不再举例了。
- `Object.setPrototypeOf(target, prototype)`方法： 用于将`target`对象的原型设置为`prototype`。例如，对于上面的`machine`对象，我们可以这样做。

  ```
  Object.setPrototypeOf(machine, Hero.prototype);
  ```

  关于修改现有对象的原型，我们稍后还会做详细讨论。
- `Object.preventExtensions(target)`方法： 用于关闭`target`对象的可扩展性，该操作是不可逆的。由于我们之前已经演示过该方法的使用，这里就不再举例了。
- `Object.isExtensible(target)`方法： 用于判断`target`对象的可扩展性，其返回`false`就代表`target`对象不可扩展了。由于我们之前已经演示过该方法的使用，这里就不再举例了。
- `Object.seal(target)`方法： 用于封印`target`对，防止其他代码删除对象的属性，这也是个不可逆的操作。由于我们之前已经演示过该方法的使用，这里就不再举例了。
- `Object.isSealed(target)`方法： 用于判断`target`对象是否已经封印，其返回`true`就代表`target`对象已被封印。由于我们之前已经演示过该方法的使用，这里就不再举例了。
- `Object.freeze(target)`方法： 用于冻结`target`对象，使得其他代码不能添加、删除或更改该任何对象属性，这也是个不可逆的操作。由于我们之前已经演示过该方法的使用，这里就不再举例了。
- `Object.isFrozen(target)`方法： 用于判断`target`对象是否已经冻结，其返回`true`就代表`target`对象已被冻结。由于我们之前已经演示过该方法的使用，这里就不再举例了。

B.1.4 原型继承机制

正如之前所说过的，我们之所以要用继承现有类的方式来创建一个新的类，主要是因为想让新建的类共享现有类的接口及其部分实现。而在没有引入类定义语法之前，JavaScript中的继承关系是以原型链的形式来呈现的。

B.1.4.1 理解原型链

以我们之前使用的`hero_1`对象为例，它的原型是`Hero.prototype`，而`Hero.prototype`的原型则是`Object.prototype`，这是JavaScript中的最终原型。在JavaScript中，这一连串的原型就被称作`hero_1`对象的“原型链”。换而言之，`hero_1`对象从一开始就已经位于一组继承关系中了，它既可以调用`Hero.prototype`中定义的接口与实现，也共享了`Object.prototype`中定义的接口与实现。例如，下面用`hero_1`对象来调用`Object`对象的实例方法。

```
console.log(hero_1.toString());        // 输出： [object Object]
console.log(hero_1.valueOf());         // 输出： Hero { name: 'owlman' }
console.log(hero_1.toLocaleString()); // 输出： [object Object]
```

如读者所见，hero_1对象也可以调用Object对象的实例方法，所以可以认为是Hero()构造函数在定义其原型的同时，"继承"了Object()构造函数定义的原型。为了便于理解，我们在某些情况下可以将Object()构造函数“看作”是Hero()构造函数的“父类”。但与此同时，读者心里必须要清楚，这种类比并不完全符合事实，构造函数至始至终都是一个有实体存在的“对象”，而不是一个代表了对象模版的“类”，这些概念在技术上这是不能混淆的。也就是说，在JavaScript中，继承关系是直接发生在对象与对象之间的，这与我们所熟悉的Java这些传统的面向对象语言有很大的不同。

当然，在这里Hero对象继承Object对象的动作是由JavaScript语言机制自动完成的，下面我们以手动调整继承关系的方式来展示一下原型链机制的灵活性。想必大家还记得之前使用的machine对象吧？它是一个用Object()构造函数定义的对象。现在，我们来改变一下它在原型链中的位置，使其变成一个由Hero()构造函数定义的对象，该怎么办呢？答案很简单，只需要将machine对象的原型设置成Hero.prototype即可，我们可以接着上面的代码这样写。

```
// 之前的定义：let machine = { name:'robot'};
Object.setPrototypeOf(machine, Hero.prototype);
// 调用 Hero.prototype 上的方法：
machine.sayHello();                    // 输出： 你好 robot
// 调用 Object.prototype 上的方法：
console.log(machine.toString());       // 输出： [object Object]
console.log(machine.valueOf());        // 输出： Hero { name: 'robot' }
console.log(machine.toLocaleString()); // 输出： [object Object]
```

是的，这里看起来会有些奇怪，我们没有创建新的对象，只是调整了一下现有对象的原型，就改变了它的数据类型。也就是说，从类型的概念上来说，machine对象现在由Object对象变成了一个Hero对象，而后者是前者的“子类”。这就是JavaScript的灵活性，这种自由度在 C++、Java 这些语言中是很难想象的。

下面，让我们稍微回归一下传统，既然构造函数在JavaScript中充当了“类”的角色，那么下面就来看看如何模拟传统的面向对象编程，基于现有的“类”来创建新的“类”。毕竟我们刚才直接让一个machine实体变成了一个Hero“类”的对象，怎么看都有些粗鲁。下面，我们来设计一个能创建机器英雄的“类”，它想必应该是个AI_Hero“类”，其定义代码如下。

```
function AI_Hero(name) {
    this.name = name;
}
AI_Hero.prototype = new Hero();
```

如读者所见，如果我们想让AI_Hero继承自Hero，只需要正常定义一个名为AI_Hero的构造函数，然后将该构造函数的prototype属性设置为一个Hero实体即可。请注意，在这里构建Hero对象是不必传递实参的，因为在AI_Hero所在作用域中，this是引用不到Hero对象的实体属性的。换句话说，在用原型链实现的继承机制中，只有被定义在prototype中的东西。下面，让我们用AI_Hero构造函数重新构建一个machine对象，看看效果是否符合我们对继承的预期。

```
let machine = new AI_Hero('Machine');
// 调用 Hero.prototype 上的方法：
machine.sayHello();                    // 输出： 你好， Machine
// 调用 Object.prototype 上的方法：
```

```
console.log(machine.toString());       // 输出: [object Object]
console.log(machine.valueOf());        // 输出: Hero { name: 'Machine' }
console.log(machine.toLocaleString()); // 输出: [object Object]
```

B.1.4.2 剥开语法糖

那么，现在的问题是ES6所带来的类定义与继承语法是否改变了JavaScript对于继承机制的实现呢？为了说明这个问题，我们需要先来回顾之前使用过的`Point`类与`colorPoint`类，并对它们进行一些分析。

```
// 类设计
class Point {
    constructor(x,y) {
        this._x = x;
        this._y = y;
    }

    printCoords() {
        console.log('坐标: ('+ this._x + ', ' + this._y + ') ');
    }

    updateCoords(x,y) {
        if ((x<0) || (y<0)) {
            console.log('坐标不能为负值! ');
            return false;
        }
        this._x = x;
        this._y = y;
    }
};

class colorPoint extends Point {
    constructor(x,y,color) {
        super(x,y);
        this._color = color;
    }

    updateColor(color) {
        this._color = color;
    }

    printCoords() {
        super.printCoords();
        console.log('颜色: ', this._color);
    }

    testSuper() { // 临时新增方法: 验证super是否为父类的prototype
        Point.prototype.temp = 10;
        console.log(super.temp);
    }
}
```

```
// 先来看看“类”在JavaScript中的实际数据类型：
console.log(typeof(Point));          // 输出： function
console.log(typeof(colorPoint));     // 输出： function

// 再来看看两者之间是否属于原型继承：：
let proto = Object.getPrototypeOf(colorPoint);
console.log(proto === Point);        // 输出： true

// 最后再来看看子类的super引用的是不是父类的prototype：
const p = new colorPoint(5,5,'红');
p.testSuper();                       // 输出： 10
```

在上述代码中，我们首先用`typeof`查看了`Point`和`colorPoint`的实际数据类型，得知了它们都是函数。然后，我们用`Object.getPrototypeOf()`方法证实了这两个类之间依然是原型继承的关系。最后，我们为`colorPoint`类临时新增了一个`testSuper()`方法。在其中，我们先在`Point.prototype`上添加了一个临时变量`temp`，然后用`super`读取到了该变量的值。这证明了子类的`super`引用的正是父类的`prototype`，这就解释了为什么`super`不能引用父类对象的自有属性（如`super._x`）。

从上述分析结果，我们可以得出一个结论：ES6新增的语法并没有改变该语言用原型链实现继承机制的事实。换句话说，ES6提供类定义与继承语法的目的仅仅是为程序员们提供一种类似于使用传统面向对象编程语言的体验，我们通常将这些在语法上提供便利，但并不改变内部实现的语言特性称之为“语法糖”。在理解语法糖的作用之后，我们就可以更灵活地使用ES6提供的语法了。例如，之前为了私有化部分数据，将自有属性定义成了构造函数的局部变量。相应地，为了让对象方法能访问这些私有数据，它们也必须在构造函数内定义。但想必大家都还记得，我们当时遇到了一个麻烦：这些对象方法都被定义在了`this`上，这不仅带来了代码冗余，而且让它的子类无法继承父类的方法。现在，我们知道了JavaScript的继承机制依然是用原型链实现的，所以只需要将这些方法定义在`Point.prototype`上即可，具体做法如下。

```
class Point {
    constructor(x,y) {
        let _x = x;
        let _y = y;

        Point.prototype.printCoords = function() {
            console.log('坐标： ('+ _x + ', ' + _y + ') ');
        };

        Point.prototype.updateCoords = function(x,y) {
            if ((x<0) || (y<0)) {
                console.log('坐标不能为负值！ ');
                return false;
            }
            _x = x;
            _y = y;
        };
    }
};

class colorPoint extends Point {
    constructor(x,y,color) {
        super(x,y);
```

```
        this._color = color;
    }

    updateColor(color) {
        this._color = color;
    }

    printCoords() {
        super.printCoords();
        console.log('颜色: ', this._color);
    }
}

// 接下来逐个测试方法的调用
const p = new colorPoint(5,5,'红');
p.printCoords();          // 输出:  坐标:  (5,5)
                          //        颜色: 红
p.updateCoords(10,10);
p.updateColor('绿');
p.printCoords();          // 输出:  坐标:  (10,10)
                          //        颜色: 绿
```

现在，`colorPoint`类对象的方法就可以通过`super`调用`Point`类定义的方法了。当然，需要提醒读者的是，在享受JavaScript赋予我们的自由度之前，需要先对自己所做的事有所把握。毕竟，类定义与继承语法在JavaScript中的出现是经历了很多年的呼吁的，所以我们虽然在原则上鼓励读者去了解这种语法背后的实现机制，但并不鼓励轻易破坏或绕过它。

B.2 异步编程

在上一节中，我们学习了面向对象编程，这种编程方式主张隐藏对象的具体实现，然后将该对象允许执行的操作以接口的形式提供给它的用户。细心的读者可能已经发现了，到目前为止，本书中所讨论的内容都是站在对象提供方的角度上思考问题的，包括将相关操作封装成函数，将相关数据组织成数据结构，然后再将数据结构与函数组合成对象，最后隐藏对象的实现并对外提供接口。但再然后呢？是不是剩下的编程工作就是按部就班地调用这些接口就行了呢？或者说，是不是只要我们实现并提供了这些接口，编程问题就解决了百分之八九十，剩下的都是一些按表操课的简单工作呢？

答案当然是否定的，事实恰恰相反，在编程的大部分工作里，我们都是在使用对象，而非提供对象。且不说在浏览器端或服务器端，我们大部分时间都在使用 jQuery、Vue 这些库或框架提供的对象来解决问题，即使之前我们在编写纯 ECMASrcipt 代码时，大部分时间也只是单纯地在使用 ES5/ES6 中定义的标准对象，例如`Array`、`Map`等。所以，如何有效地使用对象的接口才是我们在编程工作中所要面对的主要问题。

接下来，我们就要站在对象使用者的角度来思考编程问题，并以此为契机介绍一下JavaScript中最常见的编程方式：异步编程。当然了，在具体介绍异步编程之前，我们不妨先来思考一下它能为我们解决什么问题。下面，让我们回到电话交换机及其测试程序上来，先回顾一下它目前基于面向对象编程的实现代码。

```
class TelephoneExchange {
    constructor(names) {        // names 形参允许指定加入该电话交换机的初始名单
        this.mp = new Map();
        this.firstNum = 1001;     // 该电话交换机的第一个未占用的号码
```

```
        for(let name of names) {
            this.firstNum++;
            this.mp.set(this.firstNum, name); // 为初始名单分配电话号码
        }
    }

    add(name) {                             // 为新客户添加线路
        this.firstNum++;
        this.mp.set(this.firstNum, name);
    }

    delete(number) {                        // 删除线路
        this.mp.delete(number);
    }

    update(number, name) {                  // 修改已有线路的所属人
        if (this.mp.has(number)) {
            this.mp.set(number, name);
        } else {
            console.log(number + '是空号！');
        }
    }

    call(number) {                          // 拨打指定线路
        if (this.mp.has(number)) {
            let name = this.mp.get(number);
            console.log('你拨打的用户是：  ' + name);
        } else {
            console.log(number + '是空号！');
        }
    }

    callAll() {                             // 拨打所有线路
        for (let number of this.mp.keys()) {
            this.call(number);
        }
    }
};

function testTelephoneExchange(phoneExch) {
    phoneExch.callAll();
    console.log('-----------');
    phoneExch.add('owlman');
    phoneExch.callAll();
    console.log('-----------');
    phoneExch.delete(1002);
    phoneExch.callAll();
    console.log('-----------');
    phoneExch.update(1003,'batman');
    phoneExch.callAll();
    console.log('-----------');
}
testTelephoneExchange(new TelephoneExchange(['张三', '李四']));
```

如读者所见，我们目前在使用对象接口时都是一个接着一个来调用的。譬如在上面的测试过程中，对`add()`接口的调用必然会在`delete()`接口被调用之前完成，而对`delete()`接口的调用也必然会在`update()`接口之前。同样的，在`callAll()`接口的实现中，呼叫“张三”的电话必定会在“李四”之前完成。对于这种让计算机按照事先设定的任务顺序一个接着一个执行的编程方式，我们称之为同步编程。这种编程方式的优势在于它可以使整个程序的执行过程可被预期，并且不容易出现“同时添加和删除同一数据”这一类的任务逻辑问题。

但在同步编程也存在着一个很大的劣势，那就是它会让所有顺序靠后的调用都必须要等到其前一个调用完成之后才能执行。例如，在`callAll()`接口的实现中，如果我们想打“李四”的电话，就必须等“张三”的这一通电话打完之后才能进行。这如果是在真实的电话网络中（而不仅仅是在内存中执行查表操作的话），即使排除了人为因素的影响，网络本身的连通速度也要比程序执行的速度慢得多，这意味着程序在拨出“张三”电话之后，一直到与“张三”完成通话之前的这段时间里必须停下来等，在编程术语中，这种等待被称为“阻塞”状态。如果一个程序经常被置于阻塞状态，它的执行效率就会受到严重的影响。请试想一下，如果我们的“电话簿”上有几百万人，像上面这种使用同步调用的测试方案在效率上是可接受的吗？

况且，类似的问题在实际编程工作中并不少见，毕竟无论是终端设备的输入输出、磁盘文件的读写，还是数据库的存取，Web UI的响应，其速度都是远低于内存存取和 CPU 运算的。如果我们在所有场景中一律采用同步编程的方式，程序的执行效率就会成为一个严重的问题。在编程方法学上，这个问题通常是通过并发编程的方式来解决的。所谓并发编程，就是将相关的操作分别封装成独立的执行单元，这些单元之间可共享数据，但执行过程彼此独立，它的具体实现方式通常有以下两种。

- **多线程编程**： 这种编程方式主张为并发任务单独开辟一个线程，然后交由操作系统的线程机制来管理它们的执行。这样做的好处在于可以借助操作系统的现有机制来来实现计算机资源的最优化利用，坏处是会让程序员们在处理线程之间的数据同步时面临很大的挑战，一不小心就会造成死锁、竞争等各种难以处理的问题。

- **异步编程**： 这种编程方式主张将并发任务封装成可异步调用的接口，以此来避免程序过于频繁地进入阻塞状态。但是，这些异步接口是如何在既不开辟新线程，又不阻塞主线程的情况下获得执行机会的呢？或者说，这些会带来阻塞的磁盘读写、数据库存取操作是在什么时候、由谁来完成的呢？简而言之，答案就是程序的执行环境。例如在JavaScript中，其执行环境的解释引擎中会管理着一个任务队列，每当程序的主线程执行到异步调用时，就会选择将其暂存到任务队列中，并继续处理接下来的操作。待时机合适之时（由程序执行环境来判断），再回头来执行暂存在任务队列中的调用。这样做的好处是让程序员们避开了多线程编程中那些难以处理的数据同步问题，坏处是它会让代码的执行顺序难以预测，并且常常违反人们的直觉，这会给程序的调试带来不少麻烦。

众所周知，JavaScript 这门语言的设计初衷是为了赋予 Web 页面响应用户操作的能力，实现这种能力最为关键的就是要避免 Web 页面所在的线程频繁地进入阻塞状态，如果让它因经常卡顿而造成用户体验不佳的话，问题将是致命的。而对于 Web 页面来说，浏览器通常也只能允许它在一个线程内活动，因此JavaScript在相当长的一段时间里也只能采用单线程的异步编程方式来响应用户操作。后来，Node.js 运行环境与 Electron 桌面框架也都选择延续这一编程方式，它们对磁盘文件的读写、数据库的存取以及 GUI 的事件响应等会带来阻塞状态的操作也都提供了异步调用的接口。所以如今在现实环境中，但凡要用JavaScript解决一点实际问题，基本都会用到异步接口。从某种意义上来说，JavaScript 之所以能发展成今天这样一门近乎全能的编程语言，就是因为它对异步编程这种方式有一种近乎天然的强大支持。

B.2.1 异步实现方案

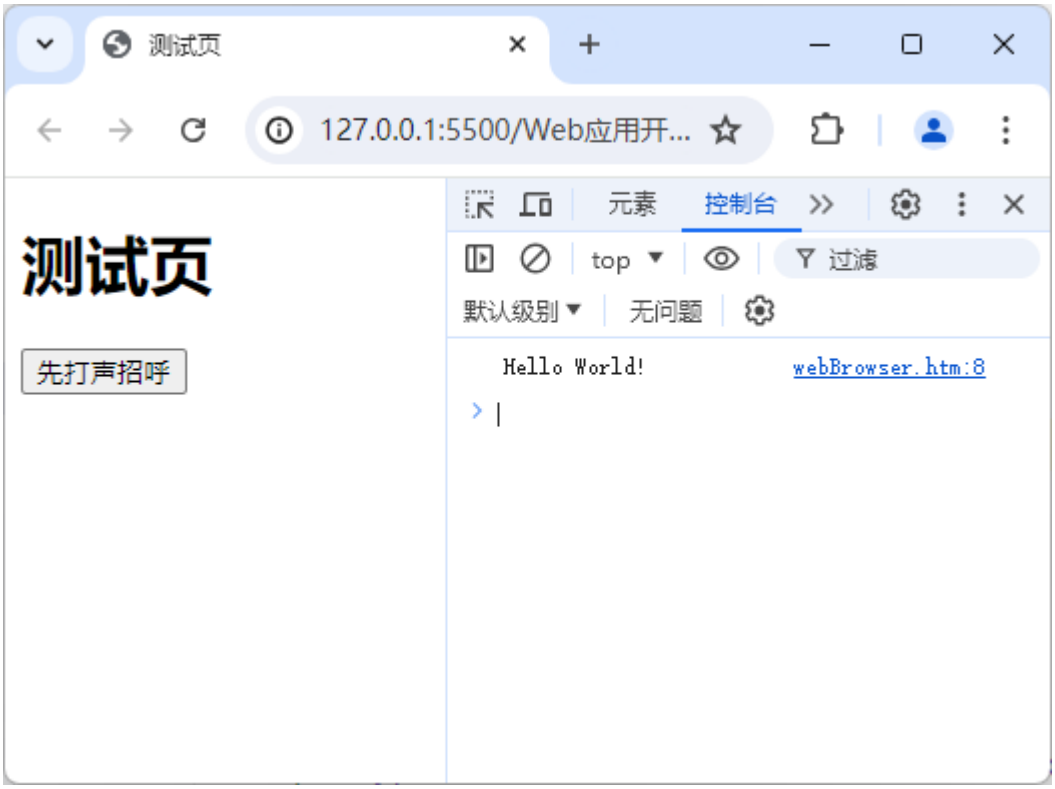
在理解了异步编程的概念以及它对于JavaScript的意义之后，我们就可以具体地来讨论一下异步编程的实现方案了。下面先从最简单的开始。

B.2.1.1 事件驱动

正如我们之前所说，JavaScript这门语言最初的作用就是在浏览器端响应用户在Web UI上的操作。而Web UI与桌面的GUI程序一样，它们对用户操作的响应能力都是基于事件驱动模型来构建的。具体做法就是：用户在Web UI上执行的鼠标单击，表单提交等操作，都会触发一个相应的“事件”，而程序员会们通常会事先为这些事件“注册”一个响应函数。例如，我们早年间在定义网页时经常会像下面这样编写HTML文档。

```
<!DOCTYPE html>
<html lang='zh-cn'>
<head>
    <meta charset="UTF-8">
    <title>测试页</title>
    <script>
        function sayHello() {
            console.log('Hello World!');
        }
    </script>
</head>
<body>
    <h1>测试页</h1>
    <input type='button' value='先打声招呼' onclick='sayHello();'>
</body>
</html>
```

如读者所见，为了响应用户对页面中按钮的单击操作，我们先在`<script>`标签中定义了一个`sayHello`函数，然后在`<input type='button'>`标签中将该函数注册成了`onclick`事件的响应函数。接下来，我们可以测试一下这个响应函数是否能正常工作：先将上述代码保存到`Examples/02_studynodejs/sayhello`目录下，并将其命名为`webBrowser.htm`文件，然后在浏览器中打开它，并单击“先打声招呼”按钮，就会在网页浏览器的JavaScript控制台中看到如图B-1所示的结果。



图B-1 浏览器端的事件驱动方案

这就是一个典型的、由事件驱动的异步实现方案。在这一方案中，我们只需要将要执行的异步函数注册给某个指定的事件，然后每当用户的操作触发该事件后，浏览器引擎中的事件监听机制就会启动对这个函数的异步调用。当然了，上面这种注册事件响应函数的编码方式会让JavaScript代码与HTML标记耦合在一起，并不利于后期的修改和维护，如今人们更多时候会选择使用获取DOM节点的方式来为页面元素注册事件响应函数，关于DOM节点的获取操作，我们会等到具体介绍浏览器端的JavaScript时再来做具体演示。目前读者暂时只需要记住，无论以什么方式注册事件的响应函数，它们执行的都是相同的异步调用机制。

当然了，使用事件驱动方案来实现异步调用也不是浏览器端独有的，在服务器端一样可以注册异步调用。下面，我们就带大家来体验一下在Web应用的服务端注册事件处理函数的步骤，具体如下。

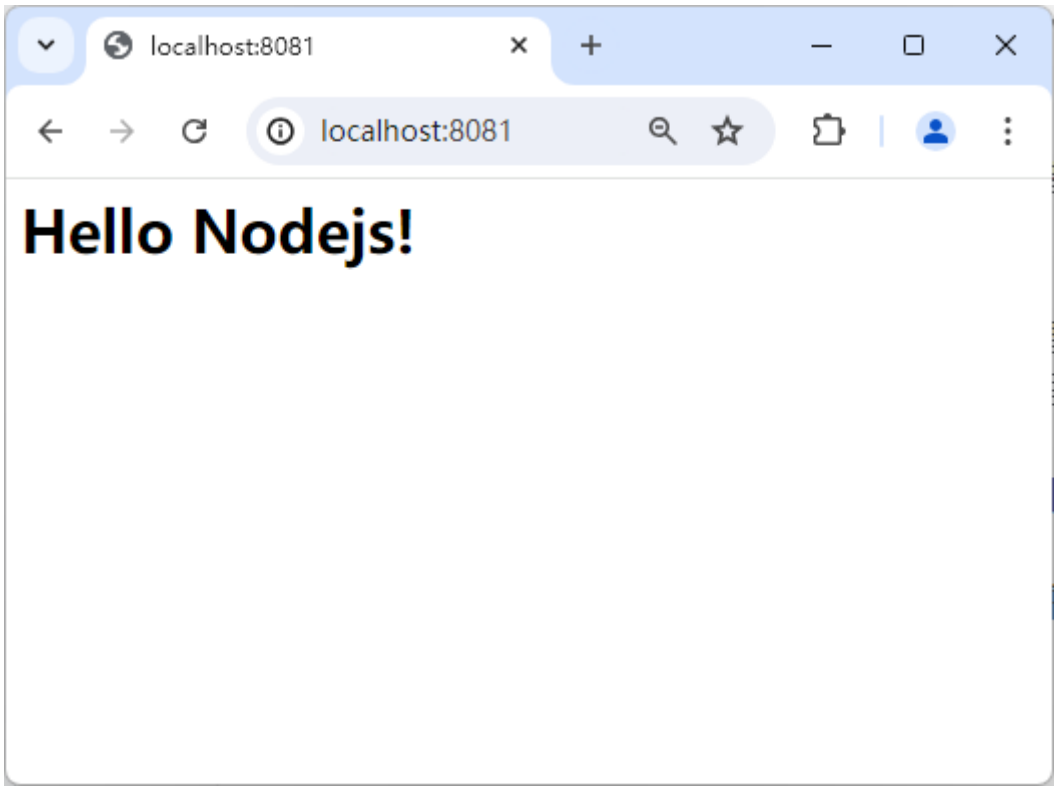
1. 在`Examples/02_studynodejs/sayhello`目录下创建一个名为`webServer.js`的脚本文件。
2. 使用代码编辑器打开`webServer.js`脚本文件，并在其中输入如下代码。

```
const http = require('http');
const server = http.createServer();

server.on('request', function(req, res){
    res.end('<h1>Hello Nodejs! </h1>');
});

server.listen(8081, function(){
    console.log('请访问http://localhost:8081/，按Ctrl+C终止服务！');
});
```

3. 在保存上述代码后，读者只需在`Examples/02_studynodejs/sayhello`目录下执行`node webServer.js`命令，并用网页浏览器访问`http://localhost:8081/`，就可以看到如图B-2所示的输出结果。



图B-2 服务器端的事件驱动方案

在上述代码中，我们用`server.on()`方法为服务器的`request`事件注册了一个事件处理函数，让服务器在接收到浏览器请求时返回`'<h1>Hello Nodejs!</h1>'`这个字符串。这里采用的就是事件驱动的异步方案。

B.2.1.2 回调函数

正如读者在上面所看到的，事件驱动的异步实现方案也会使用回调函数，但这些回调都必须要有一个“事件触发者”的角色存在，换句话说，必须要有人单击Web UI上的按钮或者向服务器发送HTTP请求，我们事先注册的事件响应函数才会被调用，并且如果这个事件被反复触发，该函数也会反复被调用。下面，我们来看一种会自动执行的、一次性的异步实现方案。例如，如果我们想让程序延时1秒钟再调用一个函数，就可以像下面这样做。

```
setTimeout(function() {
    console.log('异步操作');
}, 1000);
```

在上述代码中，我们给`setTimeout()`函数传递了两个实参,第一个实参是要延时执行的回调函数，第二个实参是要延时的具体时间，单位为毫秒。换而言之，上述调用的作用就是在1000毫秒（即1秒）之后执行输出'异步操作'字符串的函数。请注意，这里不需要任何人触发任何事件，只要过了指定的时间，回调函数就会被执行。为了证明这里执行的是一个异步调用，我们接下来在异步操作中混入一些同步操作，然后看看它们的输出顺序。

```
console.log('同步操作_1');

setTimeout(function() {
    console.log('异步操作_1');
}, 1000);
```

```
console.log('同步操作_2');

setTimeout(function() {
    console.log('异步操作_2');
}, 500);

console.log('同步操作_3');

// 以上代码输出:
// 同步操作_1
// 同步操作_2
// 同步操作_3
// 异步操作_2
// 异步操作_1
```

如读者所见，同步操作会在异步操作之前全部输出，并且由于异步操作_2设定的延时少于异步操作_1，所以它也会被先执行。如今，在时下流行的Vue.js等Web前端框架和 Node.js 运行环境中，存在着大量这种采用回调函数方案实现的异步接口，用于执行文件读写、数据库存取、网络访问等操作，它基本上成为了JavaScript的主要编程方式。

B.2.2 异步流程控制

但异步编程也有自己要面对的难题，如果一个程序充满了各种异步操作，它的流程控制就会成为一个令人头疼的问题。例如，对于下面这段代码。

```
for(let i = 0; i < 10; ++i) {
    setTimeout(function() {
        console.log('异步操作_', i);
    }, Math.random()*1000);
}
```

我们能预测它的输出顺序吗？事实上，由于循环的每轮迭代在调用`setTimeout()`函数时设定的延时都是0到1000毫秒之间的随机数，所以这段代码每次执行的输出顺序都是不同的。当然了，如果这些异步调用之间没有任何依赖关系，其执行顺序的不确定就不会带来什么影响，甚至这种随机顺序可能原本就是我们想要的效果。但如果后一个异步调用需要依赖于前一个异步调用的结果，甚至它们之间有更复杂的关系的话，那就得考虑一下异步编程的流程控制了。

B.2.2.1 回调嵌套

下面，让我们从最简单的情况开始。如果后一个异步调用依赖于前一个异步调用的结果，最简单粗暴的解决方案就是嵌套式地调用异步接口，像下面这样。

```
setTimeout(function() {
    let name = 'owlman';
    setTimeout(function(){
        console.log('Hello', name);
    }, 1000);
```

```
}, 1000);
// 以上代码输出:  Hello owlman
```

如读者所见，内层异步调用输出的内容依赖于外层异步调用中定义的`name`变量，所以它们是按照顺序执行的。但是回调嵌套毕竟不是解决异步流程控制的最佳方式，如果相互依赖的异步调用超过了3个以上，这时候再使用回调嵌套的方式来实现，就会让代码变得非常丑陋，例如像下面这个样子。

```
setTimeout(function() {
    let i = 1;
    console.log('异步操作_', i);
    ++i;
    setTimeout(function() {
        console.log('异步操作_', i);
        ++i;
        setTimeout(function() {
            console.log('异步操作_', i);
            ++i;
            setTimeout(function() {
                console.log('异步操作_', i);
                ++i;
                setTimeout(function() {
                    console.log('异步操作_', i);
                    ++i;
                }, 1000);
            }, 1000);
        }, 1000);
    }, 1000);
}, 1000);
// 以上代码输出:
// 异步操作_ 1
// 异步操作_ 2
// 异步操作_ 3
// 异步操作_ 4
// 异步操作_ 5
```

在编程方法学上，代码难看与否从来就不是一个单纯的审美问题，代码的可读性直接关系到其后期维护的难易。基本上，可读性差的代码通常会给维护工作带来地狱一般的环境，所以，我们常常将上面这种层层嵌套的回调称之为“回调地狱”。除此之外，回调嵌套的方式也解决不了一些更复杂的异步操作流程。例如，如果我们想用一个循环调用 10 个异步接口，待这10个异步操作都完成之后再输出一条消息告知用户，可能会下像面这样写。

```
for(let i = 0; i < 10; ++i) {
    setTimeout(function() {
        console.log('异步操作_', i);
    }, Math.random()*1000);
}

console.log('所有操作完成');
```

结果，我们发现无论循环内异步操作的顺序怎么变，原本应该最后输出的信息总是最先被输出。另外，我们还可能想同时调用两个异步接口，然后获取先完成的那个异步操作的结果，以此为基础来执行下一步操作。这些都是靠回调嵌套方式无法实现的。所以，我们得另辟蹊径。

B.2.2.2 异步封装

到目前为止，我们解决上述异步流程控制问题的解决思路基本都大同小异，就是设法将异步操作封装成一个可独立执行的单元（包括函数和对象），然后通过参数传递和返回值与其他操作交换数据，这样一来，这些异步操作就会按照指定的顺序来执行了。下面，就让我们来介绍在业界常见的几种用于封装异步操作的工具。

借助第三方库：

在ES6发布之前，我们通常都需要借助于各种第三方库来解决问题。例如，我们可以在 Node.js 中按以下步骤引入一个名为`async`的模块（请注意：这不是我们后面要介绍的`async/await`语法，它只是一个第三方库），然后用它解决异步操作的同步问题，其具体操作步骤如下。

1. 在`Examples/02_studynodejs/sayhello`目录下执行`npm install async`命令，将`async`库安装到当前示例目录中。
2. 在`Examples/02_studynodejs/sayhello`目录下创建一个名为`useAsync.js`的脚本文件，并在其中输入如下代码。

```
// 使用async库解决异步流程问题

const Async = require('async');
Async.series([
function (callback) {
setTimeout(function() {
    console.log('异步操作_1');
    callback();
}, Math.random()*1000);
},

function (callback) {
    setTimeout(function() {
        console.log('异步操作_2');
        callback();
    }, Math.random()*1000);
},

function (callback) {
    setTimeout(function() {
        console.log('异步操作_3');
        callback();
    }, Math.random()*1000);
    }
], function(){});
```

3. 在`Examples/02_studynodejs/sayhello`目录下执行`node useAsync.js`命令，就会看到命令行终端中陆续输出下面三个字符串。

```
异步操作_1
异步操作_2
异步操作_3
```

如读者所见，`async`库就是提供一组用于异步流程控制的函数，例如`series()`、`waterfall()`、`parallel()`等。调用这些函数都需要提供两个实参，第一个实参是一个函数列表，该列表中的每个元素都是一个以某个回调函数为参数的函数，其中封装了将要只需的异步操作；然后，这些函数的第二个实参是函数列表中每个元素要调用的回调函数。当然了，我们在之前封装的那些异步操作中也都得调用一下这个回调函数。另外需要说明的是，我们在这里只示范了`series()`函数的用法，它的作用是让异步操作按照同步顺序只需。如果读者有兴趣，也可以通过参考`async`库的官方文档 了解并实验一下该库中其他函数的用法。毕竟，在ES6提供了更好的解决方案之后，我们已经很少再使用这些第三方的解决方案了，读者只需要对它们有所了解即可。

`Promise`对象：

在ES6发布之后，JavaScript核心组件中就有了属于自己的用于封装异步操作的解决方案：`Promise`对象。从使用方式上来说，我们可以将其视为一种用于存放异步操作的容器，它会根据以下三种状态来执行异步操作。

- `pending`：这是被封装的异步操作还尚未被执行时的状态，由于`Promise`对象一经创建就会立即被执行，所以我们可以认为这是`Promise`对象的初始状态。
- `fulfilled`：如果`Promise`对象中所封装的异步操作在执行过程中没有发生错误，或者抛出异常，就会进入“fulfilled”状态，代表异步操作执行成功。
- `rejected`： 如果`Promise`对象所封装的异步操作在执行过程中发生了错误，或者抛出了异常，就会进入“rejected”状态，代表异步操作执行失败。

`Promise`对象的这三种状态之间的转换只取决于其内部封装的异步操作，不受任何外部因素的干扰。并且在这三种状态中，只有“pending”状态可以转换成“fulfilled”和“rejected”这两种状态的其中一种，而且这种状态转换是不可逆的。也就是说，一个“pending”状态的`Promise`对象在转换为“fulfilled”状态之后，就不能再回到“pending”状态，也不可能再切换到“rejected”状态，反之亦然。下面，我们来看一下`Promise`这个构造函数的具体使用语法。

```
[let 或 const] [对象名] = new Promise(function(resolve, reject) {
    if([异步操作成功]) {
      resolve([待处理的数据]);
    } else {
      reject([错误信息]);
    }
});
[对象名].then(function([待处理的数据]{
    [处理数据]
}), function([错误信息]) {
    [处理错误]
})
```

如读者所见，`Promise`构造函数只接受一个回调函数为实参，该函数定义的是`Promise`对象的执行器。该执行器又会接受两个回调函数为实参，其中：`resolve`定义的是对象转换为“fulfilled”状态时调用的函数，而`reject`则定义了对象转换为“rejected”状态时调用的函数。然后，我们会调用被创建对象的`then()`方法分别将`resolve`和`reject`对应的函数实参传递进去，下面来看一个具体示例。

```
function asyncOperator(number) {
    return new Promise(function(resolve,reject) {
        setTimeout(function() {
            if(number < 0) {
                reject('编号不能是负数！');
                return;
            }
            resolve(number);
        }, Math.round()*1000);
    });
}

asyncOperator(1).then(function(number) {
    console.log('异步操作_', number);
}, function(err) {
    console.log(err);
}); // 输出： 异步操作_ 1

asyncOperator(-1).then(function(number) {
    console.log('异步操作_', number);
}, function(err) {
    console.log(err);
});// 输出： 编号不能是负数！
```

在上述代码中，`asyncOperator()`函数返回了一个`Promise`对象，该对象中封装的异步操作会在`number`为负数时调用`reject()`，不然就调用`resolve()`。然后我们调用了`asyncOperator()`函数返回对象的`then()`方法，并为其指定了`resolve`和`reject`实参。当然，调用`then()`方法的实参是可选的，当我们不想处理“rejected”状态或者“fulfilled”状态时，也可以分别将其调用如下。

```
asyncOperator(1).then(function(number) {
    console.log('异步操作_', number);
}); // 输出： 异步操作_ 1

asyncOperator(-1).then(null, function(err) {
    console.log(err);
}); // 输出： 编号不能是负数！
```

甚至对于“rejected”状态，`Promise`对象还有个专用的`catch()`方法，例如下面两个调用的效果是完全一致的。

```
asyncOperator(-1).then(null, function(err) {
    console.log(err);
}); // 输出： 编号不能是负数！
```

```
asyncOperator(-1).catch(function(err) {
    console.log(err);
}); // 输出： 编号不能是负数!
```

在了解了Promise对象的基本用法之后，接下来，就让我们来解决一下之前提出来的异步流程控制问题。首先是回调地狱问题，想必大家还记得，造成多层回调嵌套的原因是因为后一个异步调用需要前一个异步调用提供的数据，例如下面在这个三层的回调嵌套中。

```
const obj = {};
setTimeout(function() {
    obj.name = 'batman';
    setTimeout(function() {
        obj.sayHi = function() {
            console.log('hello', this.name);
        };
        setTimeout(function() {
            obj.sayHi();
        }, 1000);
    }, 1000);
}, 1000);
// 以上代码输出： hello batman
```

第三层回调函数中使用了前两层回调所构建的对象，下面我们要使用Promise对象实现相同的功能，这其中的关键就是要利用then()方法的返回值，先来看一下代码。

```
function asyncOperator(obj) {
    return new Promise(function(resolve,reject) {
        setTimeout(function() {
            if(obj === undefined) {
                reject('对象未定义！');
                return;
            }
            resolve(obj);
        }, 1000);
    });
}

asyncOperator(new Object())
  .then(function(obj) {
    obj.name = 'batman';
    return asyncOperator(obj);
  })
  .then(function(obj) {
    obj.sayHi = function() {
        console.log('hello', this.name);
    };
    return asyncOperator(obj);
  })
```

```
  .then(function(obj) {
    obj.sayHi();
  })
  .catch(function(err) {
    console.log(err);
  });
// 以上代码输出： hello batman
```

读者现在看到的就是一个`Promise`对象的`then()`方法调用链，之所以能这样使用，是因为`then()`方法无论如何都会返回一个新的`Promise`对象。当然了，具体返回一个怎样的`Promise`对象，会因我们具体调用它的方式存在着一些不同，具体如下。

- 如果我们不具体指定返回值，`then()`方法会返回一个空的Promise对象，例如：

  ```
  const pobj = asyncOperator(10).then(function(obj){});
  console.log(pobj); // 输出: Promise { <pending> }
  pobj.then(function(){
      console.log('pobj.then'); // 输出: pobj.then
  })
  ```

- 如果我们指定其返回一个基本类型的值，`then()`方法会将该值传递给新的`Promise`对象的`resolve()`处理函数，例如：

  ```
  asyncOperator(10)
  .then(function(obj) {
      return ++obj;
  })
  .then(function(obj) {
      console.log(obj); // 输出: 11
  });
  ```

- 如果我们指定其返回一个具体的`Promise`对象，`then()`方法就会根据这个对象来执行操作，就像读者在上面那个解决回调地狱的示例中看到的那样。

- 除此之外，`then()`方法还能返回一个定义了`then()`方法的对象，我们将其称作`thenable`对象，例如：

```
asyncOperator(new Object())
.then(function(obj){
    obj.then = function(resolve, reject){
       resolve('batman');
    };
    return obj;
})
.then(function(msg) {
    console.log(msg); // 输出: batman
});
```

在了解了如何避免回调嵌套导致的回调地狱问题之后，我们再来解决一下如何在循环调用若干个异步调用之后再输出一条提示信息，还记得吗？我们当时的代码是像下面这样写的。

```
for(let i = 0; i < 10; ++i) {
    setTimeout(function() {
        console.log('异步操作_', i);
    }, Math.random()*1000);
}

console.log('所有操作完成');
```

结果，由于循环中执行的都是异步的延时调用，“所有操作完成”这条信息始终会第一个输出。如何让最后一条信息等待循环中所有异步调用完成之后再输出呢？答案是使用`Promise.all()`方法，具体如下。

```
function asyncOperator(number) {
    return new Promise(function(resolve, reject){
        setTimeout(function() {
            console.log('异步操作_', number);
            resolve();
        }, Math.random()*1000);
    });
}

let promises = new Array();
for(let i = 0; i < 10; ++i) {
    promises.push(asyncOperator(i));
}

Promise.all(promises)
.then(function() {
    console.log('所有操作完成');
});
```

如读者所见，我们首先将要循环调用的异步操作封装成了一个函数，然后用循环生成了一个拥有十个元素的`Promise`对象数组。然后，`Promise.all()`方法会接受一个`Promise`对象的数组为实参，并在数组中所有`Promise`对象都完成各自的异步操作之后返回一个新的Promise对象，并调用它的`then()`方法。现在，我们只需要在终端执行一下这段代码，就会看到无论异步操作的顺序如何，“所有操作完成”这条信息始终是最后一个输出。

当然，在上面执行的这些`Promise`对象中，`resolve()`处理函数只是个占位符，它不执行任何操作，但如果这些被封装的异步操作需要将某些参数传递出来怎么办？答案是：`Promise.all()`方法返回的这个`Promise`对象中会生成一个数组用于存储每个异步操作传递出来的数据，并将其交由自身的`then()`方法来处理，下面让我们来修改一下上面的代码，以便验证一下这个答案。

```
function asyncOperator(number) {
    return new Promise(function(resolve,reject){
        setTimeout(function() {
```

```
            resolve('异步操作_' + number);
        }, Math.random()*1000);
    });
}

let promises = new Array();
for(let i = 0; i < 10; ++i) {
    promises.push(asyncOperator(i));
}

Promise.all(promises)
.then(function(data) {
    for(const item of data) {
        console.log(item);
    }
    console.log('所有操作完成');
});
// 以上代码输出:
// 异步操作_0
// 异步操作_1
// 异步操作_2
// 异步操作_3
// 异步操作_4
// 异步操作_5
// 异步操作_6
// 异步操作_7
// 异步操作_8
// 异步操作_9
// 所有操作完成
```

如读者所见，`data`数组中存储了所有异步操作传递给`resolve()`处理函数的数据，我们可以在`Promise.all()`方法返回的`Promise`对象中对其进行一并处理，只不过这样做的话，所有异步操作的结果也会按顺序输出，这似乎不符合我们的设计初衷。

另外，只要`Promise`对象数组中有一个对象进入“rejected”状态，`Promise.all()`方法返回的`Promise`对象就会是“rejected”状态。例如：

```
// asyncOperator()函数的定义不变

let promises = new Array();
promises.push(Promise.reject('操作失败了'));
// 添加已处于“rejected”状态的`Promise`对象
for(let i = 0; i < 10; ++i) {
    promises.push(asyncOperator(i));
}

Promise.all(promises)
.then(function(data) {
    for(const item of data) {
        console.log(item);
    }
```

```
    console.log('所有操作完成');
})
.catch(function(err) {
    console.log(err);
});
// 以上代码输出： 操作失败了
```

在这里，`Promise.reject()`方法的作用是直接生成并返回一个已是“rejected”状态的`Promise`对象，并将“操作失败了”这条报错信息传递给了其生成对象的`reject()`处理函数。现在，我们把这个对象添加为`promises`数组的第一个元素，这会让`Promise.all()`方法忽略掉后续的所有`Promise`对象，直接返回一个“rejected”状态的`Promise`对象，并交由其`catch()`方法输出报错信息。（同样地，我们也可以用`Promise.resolve()`方法的作用是直接生成并返回一个已是“fulfilled”状态的`Promise`对象，并相关参数直接传递给其`resolve()`处理函数。）

下面，我们来解决最后一个问题：如果想在若干个异步操作中找到最先完成的是哪一个操作，并获取该操作的结果，该如何做呢？答案是使用`Promise.race()`方法，具体如下。

```
// asyncOperator()函数的定义不变

let promises = new Array();
// promises.push(Promise.reject('操作失败了'));
for(let i = 0; i < 10; ++i) {
    promises.push(asyncOperator(i));
}

Promise.race(promises)
.then(function(data) {
    console.log('最先完成的是: ', data);
})
.catch(function(err) {
    console.log(err);
});
```

如读者所见，`Promise.race()`方法的用法与`Promise.all()`方法基本相同，都接受一个`Promise`对象的数组为实参，并返回一个新的`Promise`对象。区别是，`promises`数组中只有一个元素完成了异步操作，`Promise.race()`方法就会停止执行，并将该元素的信息传递给其返回的`Promise`对象。当然了，需要注意的是，这里所谓的“最先完成的`Promise`对象”，既可以是“fulfilled”状态的，也可以是“rejected”状态的，如果我们同样将上面这个已处于“rejected”状态的`Promise`对象加到`promises`数组中，代码也会输出“操作失败了”这条报错信息。

B.2.2.3 专用语法

读者可能已经发现了，无论是采用第三方库还是使用`Promise`对象的`then()`方法调用链，都会涉及到至少三层回调，虽然这些回调都是以参数和返回值的形式来实现的，但依然不够直观，相比同步操作，它们出错的概率与维护的难度依然很高。于是，ES7在`Promise`对象基础上进一步推出了`async/await`语法，用于专门处理异步调用。

使用`async/await`语法：

下面，我们就先来示范一下这种语法的具体使用。

```
// asyncOperator()函数的定义不变

async function test() {
    const msg = await asyncOperator(1);
    console.log(msg);
}

test(); // 输出：异步操作_1
```

在上述代码中，我们在`test()`函数的定义前面加上了一个`async`关键字。该关键字的作用是告诉JavaScript解释器：这是一个要调用`Promise`对象的函数。然后在`test()`函数中，我们用`await`关键字来调用`asyncOperator()`，这时候，JavaScript解释器就会停下来等`Promise`对象执行完成，并直接将其传递给`resolve()`处理函数的数据返回给`msg`变量。等这一切操作都完成之后，再继续执行后面的代码。这样做的好处在于，我们可以直接在当前函数中处理异步操作的结果，不用使用`then()`方法调用链再传递回调函数了。如果我们用这种语法来调用一下`Promise.all()`和`Promise.race()`这两种方法，就会发现代码简单、直观了不少，例如像下面这样。

```
// asyncOperator()函数的定义不变

async function test() {
    let promises = new Array();
    // promises.push(Promise.reject('操作失败了'));
    for(let i = 0; i < 10; ++i) {
        promises.push(asyncOperator(i));
    }

    console.log('执行Promise.all()');
    const data = await Promise.all(promises);
    for(const item of data) {
        console.log(item);
    }
    console.log('执行Promise.race()');
    const msg = await Promise.race(promises);
    console.log('最先完成的是：', msg);
}

test();
```

但是，这里还是有一个问题没有解决。即这里处理的都是“fulfilled”状态的`Promise`对象，如果我们将已经处于“rejected”状态的`Promise`对象添加到`promises`数组中，该如何处理传递给`reject()`处理函数的报错信息呢？答案是使用JavaScript的异常处理机制 —— `try/catch`语句来处理。例如，我们可以像下面这样做。

```
// asyncOperator()函数的定义不变

async function test() {
```

```
    let promises = new Array();
    promises.push(Promise.reject('操作失败了'));
    // 添加已处于“rejected”状态的`Promise`对象
    for(let i = 0; i < 10; ++i) {
        promises.push(asyncOperator(i));
    }

    try {
        console.log('执行Promise.all()');
        const data = await Promise.all(promises);
        for(const item of data) {
            console.log(item);
        }
        console.log('执行Promise.race()');
        const msg = await Promise.race(promises);
        console.log('最先完成的是：', msg);
    } catch(err) {
        console.log(err);
    }
}

test();
```

如读者所见，我们将主要的执行代码放在了`try`执行块中，一旦执行过程中遇到了处于“rejected”状态的`Promise`对象，或者由其他因素产生的异常，它们传递出来的信息就会被后面的`catch`执行块捕获。在这里，"操作失败了"这条报错信息原本是传递给`reject()`处理函数的，现在将传递给`catch`执行块的`err`形参。除此之外，JavaScript中的错误信息还可以由`throw`语句来传递给`catch`执行块，例如像下面这样。

```
try {
    // 执行相关代码
    let isErr = true;
    if(isErr) {
        throw '错误信息';
    }
    // 继续执行后续代码
} catch(err) {
    console.log(err); // 输出：错误信息
}
```

B.3 综合练习

在学习了异步编程的相关知识之后，我们在本章结束之前还是照例要综合演示一下在这一章中学习到的知识点，顺便解决一下本章开头提出的问题，即如何将电话接通的延时因素考虑到我们之前实现的电话交换机及其测试程序中。首先，我们要修改电话交换机对象的`call()`方法，具体如下。

```
call(number) {                          // 拨打指定线路
    const me = this;
    return new Promise(function(resolve, reject) {
```

```
        const time = Math.random()*5000;
        setTimeout(function() {
            if (me.map.has(number)) {
                let name = me.map.get(number);
                if(time > 3000) {
                    resolve('呼叫超时');
                } else {
                    resolve('你拨打的用户是:  ' + name);
                }
            } else {
                resolve(number + '是空号! ');
            }
        }, time);
    }).then(function(msg) {
        console.log(msg);
    });
}
```

如读者所见，由于我们手里并没有物理的电话网络，所以只能使用`setTimeout()`函数来模拟电话的延时。我们将电话被允许的延时设置在了 0 到 3 秒之间，超过 3 秒则被认为超时，而实际产生的时间则是 0 到 5 秒之间的随机数，这样就制造了一定的超时概率。由于这是个异步操作，所以我们将其封装在了一个`Promise`对象中。需要注意的是，由于我们不希望一条线路的问题，影响到其他线路的测试终止，所以我们将超时和空号的情况，也一并交给`resolve()`处理函数来输出。接下来，我们继续来修改`callAll()`方法。

```
async callAll() {
    console.log('-----开始测试系统所有线路------');
    const promises = new Array();                        // 拨打所有线路
    for(let number of this.map.keys()) {
        promises.push(this.call(number));
    }
    return await Promise.all(promises).then(function() {
        console.log('-----系统全部线路测试结束------');
    });
}
```

在这里，我们将所有线路的呼叫操作都添加到一个`Promise`数组中，然后用`Promise.all()`方法来执行它们。当然，如读者所见，我们在这里使用了`async/await`语法。下面，我们还要用同样的语法来修改一下测试函数。

```
async function testTelephoneExchange(phoneExch) {
    await phoneExch.callAll();
    phoneExch.add('owlman');
    await phoneExch.callAll();
    phoneExch.delete(1002);
    await phoneExch.callAll();
    phoneExch.update(1003,'batman');
    await phoneExch.callAll();
}
```

这样一来，电话交换机及其测试程序的第四版实现方案就完成了，下面是它的全部代码：

```
class TelephoneExchange {
    constructor(names) {        // names 形参允许指定加入该电话交换机的初始名单
        this.map = new Map();
        this.firstNum = 1001;     // 该电话交换机的第一个未占用的号码

        for(let name of names) {
            this.firstNum++;
            this.map.set(this.firstNum, name); // 为初始名单分配电话号码
        }
    }

    add(name) {                           // 为新客户添加线路
        this.firstNum++;
        this.map.set(this.firstNum, name);
    }

    delete(number) {                      // 删除线路
        this.map.delete(number);
    }

    update(number, name) {                // 修改已有线路的所属人
        if (this.map.has(number)) {
            this.map.set(number, name);
        } else {
            console.log(number + '是空号！');
        }
    }

    call(number) {                        // 拨打指定线路
        const me = this;
        return new Promise(function(resolve, reject) {
            const time = Math.random()*5000;
            setTimeout(function() {
                if (me.map.has(number)) {
                    let name = me.map.get(number);
                    if(time > 3000) {
                        resolve('呼叫超时');
                    } else {
                        resolve('你拨打的用户是：  ' + name);
                    }
                } else {
                    resolve(number + '是空号！');
                }
            }, time);
        }).then(function(msg) {
            console.log(msg);
        });
    }
```

```
    async callAll() {
        console.log('-----开始测试系统所有线路------');
        const promises = new Array();                        // 拨打所有线路
        for(let number of this.map.keys()) {
            promises.push(this.call(number));
        }
        return await Promise.all(promises).then(function() {
            console.log('-----系统全部线路测试结束------');
        });
    }
};

async function testTelephoneExchange(phoneExch) {
    await phoneExch.callAll();
    phoneExch.add('owlman');
    await phoneExch.callAll();
    phoneExch.delete(1002);
    await phoneExch.callAll();
    phoneExch.update(1003,'batman');
    await phoneExch.callAll();
}

testTelephoneExchange(new TelephoneExchange(['张三', '李四', '王五', '赵六']));
```

现在，我们可以重复执行几次这个程序，看看效果是否更接近实际电话网络的测试了，如图B-3所示的是其中一次的执行结果。

```
❯ node .\TelephoneExchange\main.js
-----开始测试系统所有线路------
你拨打的用户是： 赵六
呼叫超时
呼叫超时
呼叫超时
-----系统全部线路测试结束------
-----开始测试系统所有线路------
你拨打的用户是： 赵六
你拨打的用户是： 李四
你拨打的用户是： owlman
你拨打的用户是： 王五
呼叫超时
-----系统全部线路测试结束------
-----开始测试系统所有线路------
你拨打的用户是： 赵六
呼叫超时
呼叫超时
呼叫超时
-----系统全部线路测试结束------
-----开始测试系统所有线路------
你拨打的用户是： 赵六
你拨打的用户是： batman
你拨打的用户是： owlman
你拨打的用户是： 王五
-----系统全部线路测试结束------
```

图B-3 在程序中加入异步操作